Prompt Verification Checklist

Use this physical or digital checklist to verify the outputs generated during testing in Google AI Studio. Every section must satisfy all constraints before finalizing the system prompts.

Use this checklist to verify the outputs you saved during your Google AI Studio testing sessions. Each section highlights the specific things you must look out for to confirm that the agent system prompts are behaving exactly as expected.

---

1. Research Agent Output Verification

Open your saved **Research Report** file and verify the following:

- [ ] **Title**: Starts with exactly `# Research Report: FreelanceFlow`.
- [ ] **Problem Space**: Contains 2-3 paragraphs of high-quality analysis. Verify that it mentions the specific pain points of freelancers (such as manual time tracking, invoice disputes, and tracking outstanding payments) rather than generic business jargon.
- [ ] **Existing Solutions & Competitors Table**:
  - Contains a markdown table with at least 4 rows.
  - Every competitor listed must be a real product (e.g., *FreshBooks*, *Harvest*, *Toggl*, *Wave*, or *Bonsai*).
  - The strengths and weaknesses must mention specific feature designs (e.g., "Toggl lacks built-in invoicing" or "FreshBooks gets expensive for many clients").
- [ ] **Target Users**:
  - Persona title is clear (e.g., "Solo Freelancer" or "Agency Owner").
  - Has a numbered list of at least 3 distinct pain points.
  - Mentions real workarounds currently used (e.g., manual Excel spreadsheets, Google Calendar entries, paper notebooks).
- [ ] **Technical Landscape**: Mentions APIs and libraries relevant to the product (e.g., Stripe API, PDFKit/ReportLab for PDF generation, SendGrid/SMTP for emails).
- [ ] **Key Risks**: Contains at least 2 technical risks, 2 market risks, and 2 execution risks (total 6+ numbered items).
- [ ] **Research Confidence Score**: Ends with a score (High, Medium, or Low) and a justification paragraph.
- [ ] **Word Count**: Total document size is between 900 and 1,400 words.

---

2. Requirements Agent Output Verification

Open your saved **Requirements Document** and verify the following:

- [ ] **Title**: Starts with exactly `# Requirements Document: FreelanceFlow`.
- [ ] **Functional Requirements**:
  - Contains a numbered list of at least 12 requirements.
  - Every single requirement must start with: **"The system shall..."**
  -

No placeholders: No items like *"The system shall do other tasks as needed."*

- [ ] **Non-Functional Requirements**: Covers exactly these 5 areas with measurable limits:

1. *Performance* (e.g., "load dashboard under 2 seconds")
2. *Security* (e.g., "passwords hashed with bcrypt", "JWT for authentication")
3. *Scalability* (e.g., "support up to 10,000 monthly active users")
4. *Reliability* (e.g., "99.9% service uptime target")
5. *Accessibility* (e.g., "comply with WCAG 2.1 Level AA")

- [ ] **User Stories**:
  - Contains at least 8 user stories.
  - Grouped under `###` subheadings indicating clear feature areas (e.g., `### Time Tracking`).
  - Every story strictly follows: **"As a [user type], I want to [action], so that [benefit]."**
- [ ] **Out of Scope**: Lists at least 5 clear features that will *not* be built in v1 to prevent scope creep.
- [ ] **Tech Stack Recommendation**: Lists exactly 7 layers (Frontend, Backend, Database, Auth, File Storage, Hosting, Third-Party APIs) with a specific reason *why* they fit this project, and an alternative that was considered but rejected.
- [ ] **Tech Stack JSON**:
  - Located at the absolute end of the file.
  - Contains **valid JSON** with the keys: `"frontend"`, `"backend"`, `"database"`, `"auth"`, `"hosting"`, `"key_libraries"`.
  - **Crucial**: There must be no extra text, characters, or markdown fences (` ``` `) below this JSON block.

3. Architecture Agent Output Verification

Open your saved **Architecture Blueprint** and verify the following:

- [ ] **Title**: Starts with exactly `# Architecture Blueprint: FreelanceFlow`.
- [ ] **Folder Structure**:
  - Shown as a code block.
  - Every file must be named specifically according to the feature set (e.g., `invoices.py`, `DashboardPage.jsx`).
  - **No shortcuts**: Absolutely no `...`, `[etc]`, or `[more files here]` placeholders.
- [ ] **Database Schema**:
  - SQL `CREATE TABLE` scripts are present for every table (`users`, `clients`, `projects`, `time_entries`, `invoices`, `payments`).
  - Contains a valid Mermaid ER diagram (`erDiagram` block).
- [ ] **API Endpoints**:
  - A markdown table listing at least 15 endpoints.
  - Columns match: `Method | Path | Auth Required | Request Body | Response | Description`.
  - Covers all aspects: Authentication, Clients, Projects, Time Entries, Invoices, Payments.
- [ ] **Component Hierarchy**: React component tree represented as an indented list showing shared vs. page-specific components.
- [ ]

Data Flow: Contains a valid Mermaid flowchart (`flowchart TD` block) visualizing user creation/read flows and auth login flow.

- [ ] **Security Approach**: Details JWT storage, authorization, input validation, rate limiting, and CORS.
- [ ] **Environment Variables**: Table detailing all required variables, descriptions, and mock values.

4. Planning Agent Output Verification

Open your saved **Implementation Plan** and verify the following:

- [ ] **Formatting**: The output must consist **only** of a raw JSON array.
  - Starts with `[` and ends with `]`.
  - No markdown formatting fences (no ` ```json ` at the start or ` ``` ` at the end).
  - No conversational text or headers.
- [ ] **JSON Syntax**: The array parses successfully as valid JSON (verify via jsonlint.com).
- [ ] **Keys**: Every object inside the array has exactly these 7 keys:
  - `"id"`
  - `"phase"`
  - `"filename"`
  - `"filepath"`
  - `"description"`
  - `"requires"`
  - `"context_sections"`
  - `"estimated_complexity"`
- [ ] **ID Format**: Matches `{phase_prefix}_{three_digit_number}` (e.g., `db_001`, `be_003`, `fe_012`, `dv_002`).
- [ ] **Phase Names**: Must be exactly one of: `"database"`, `"backend"`, `"frontend"`, or `"devops"`.
- [ ] **Order and Dependencies**:
  - All `db_` tasks appear first (no dependencies or only depend on other `db_` tasks).
  - All `be_` tasks appear next (depend on `db_` tasks).
  - All `fe_` tasks appear third (depend on `be_` tasks).
  - All `dv_` tasks appear last.

5. Coder Agents Output Verification (Database, Backend, Frontend)

Open the code files you generated during Step 5 and verify the following:

Database Models (`models.py`)

- [ ] **ORM Pattern**: Uses SQLAlchemy 2.0 `DeclarativeBase` style, not the legacy `declarative_base()`.
- [ ] **Keys & Audits**: Every model has a UUID primary key, explicit `__tablename__`, and server-default `created_at` / `updated_at` columns.
- [ ] **Relationships**: Both sides of every relationship specify `back_populates` (no legacy `backref`).

Backend Router (`invoices.py` / controllers)

- [ ] **Imports**: Uses relative imports (e.g., `from ..models import Invoice`), not absolute package paths.
- [ ] **Errors**: Includes proper FastAPI `HTTPException` calls with descriptive error messages.
- [ ] **Dependency Injection**: Injects database session via `Depends(get_db)`, never instantiates session makers directly.
- [ ] **Documentation**: Every route has a Python docstring.

Frontend Component (`InvoicesPage.jsx`)

- [ ] **Styling**: Uses only Tailwind CSS utility classes. No inline styles.
- [ ] **API Calls**: Uses `axios` and imports the base configurations from `src/lib/api.js` (no hardcoded localhost URLs).
- [ ] **Safety**: Uses optional chaining (`?.`) and nullish coalescing (`??`) for all API response data access.
- [ ] **Export**: Features a default export.

6. QA Agent Output Verification

Open your saved **QA Report** and verify the following:

- [ ] **Title**: Starts with `# Quality Assurance Report: FreelanceFlow`.
- [ ] **Required Sections**: Contains exactly these sections in order:
  - `## Critical Issues`
  - `## Warnings`
  - `## Security Findings`
  - `## Missing Pieces`
  - `## Summary`
- [ ] **Specificity**: Does not contain vague reports. Every issue must cite a file path, line number, issue description, and suggested code fix.
- [ ] **Summary Counts**: Displays exact counts of issues and a rating (`Needs Work` / `Acceptable` / `Good`) with justification.

7. DevOps Agent Output Verification

Open your saved DevOps files (`docker-compose.yml`, `Dockerfile`, `README.md`) and verify the following:

- [ ] **No Code Fences**: The files are output as raw contents, not wrapped in ` ``` ` markdown code blocks.
- [ ] **Backend Dockerfile**: Uses Python slim, sets up and runs as a non-root user, and copies `requirements.txt` before the application code.
- [ ] **Docker Compose**: Sets up services for `backend`, `frontend`, and `db`, and sets up health checks and named volumes.
- [ ] **README**: Contains clean markdown with installation steps, prerequisites, and a development flow.